

LAPORAN
BENGKEL PEMROGRAMAN FRAMEWORK



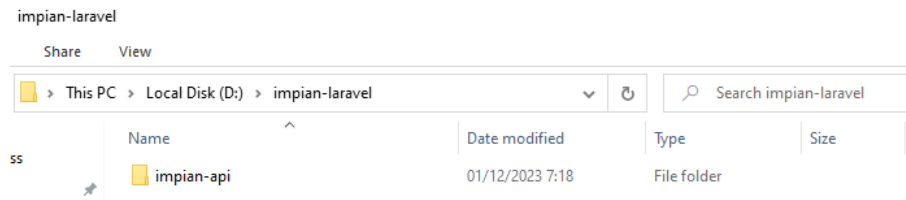
Dosen/PLP :
Mutia Sari Zulvi, S.S.T., M.M.S.I
Rufina Pramudita

Disusun oleh :
Impian Aditya Nanda
2 TI F

PRODI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI

Backend Laravel CRUD

1. Saya membuka folder proyek dengan Visual Studio Code (File -> Open Folder).



2. Setelah itu, saya membuka Terminal di dalam VSCode dan menjalankan perintah **php artisan serve**.

A screenshot of a VS Code terminal window. The terminal shows the following commands and output:

```
PS D:\impian-laravel\impian-api> php intesan serve
Could not open input file: intesan
PS D:\impian-laravel\impian-api> php artisan serve

[INFO] Server running on [http://127.0.0.1:8000].

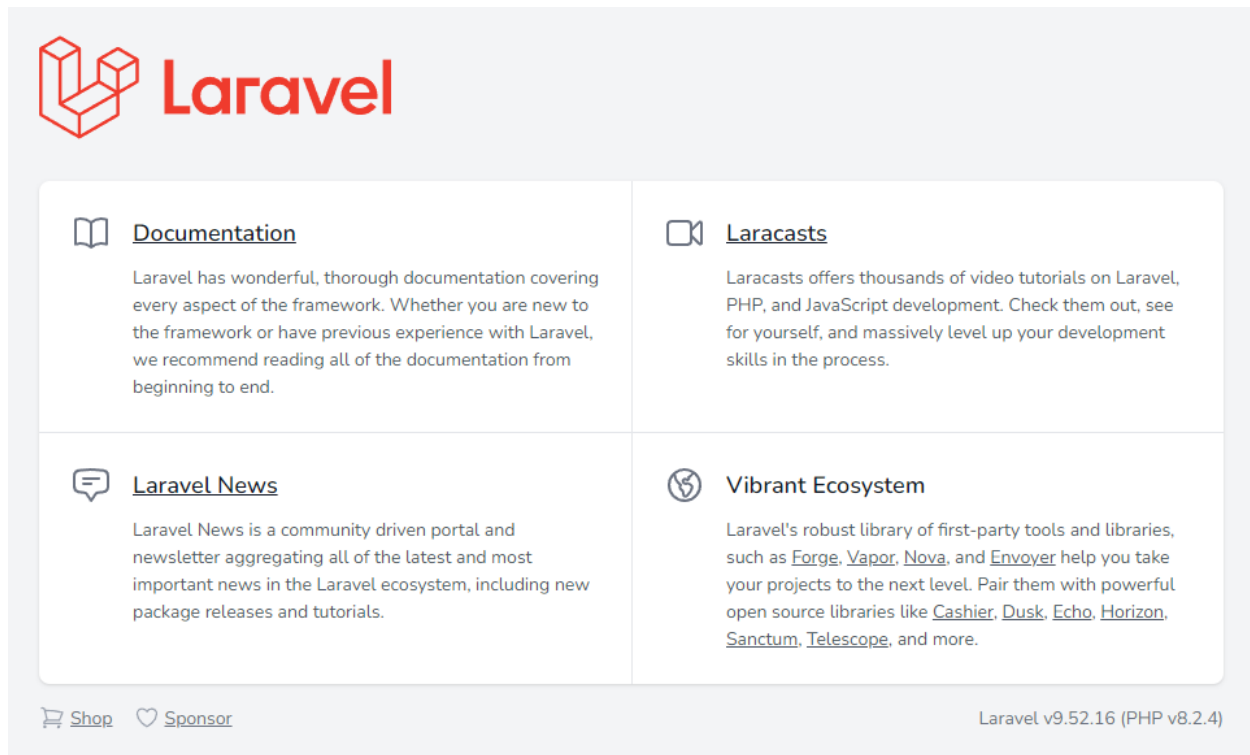
Press Ctrl+C to stop the server

2023-12-01 07:24:39 .....
..... ~ 0s
2023-12-01 07:24:39 /favicon.ico .....
..... ~ 0s
2023-12-01 07:28:28 .....
..... ~ 0s
```

Analisa :

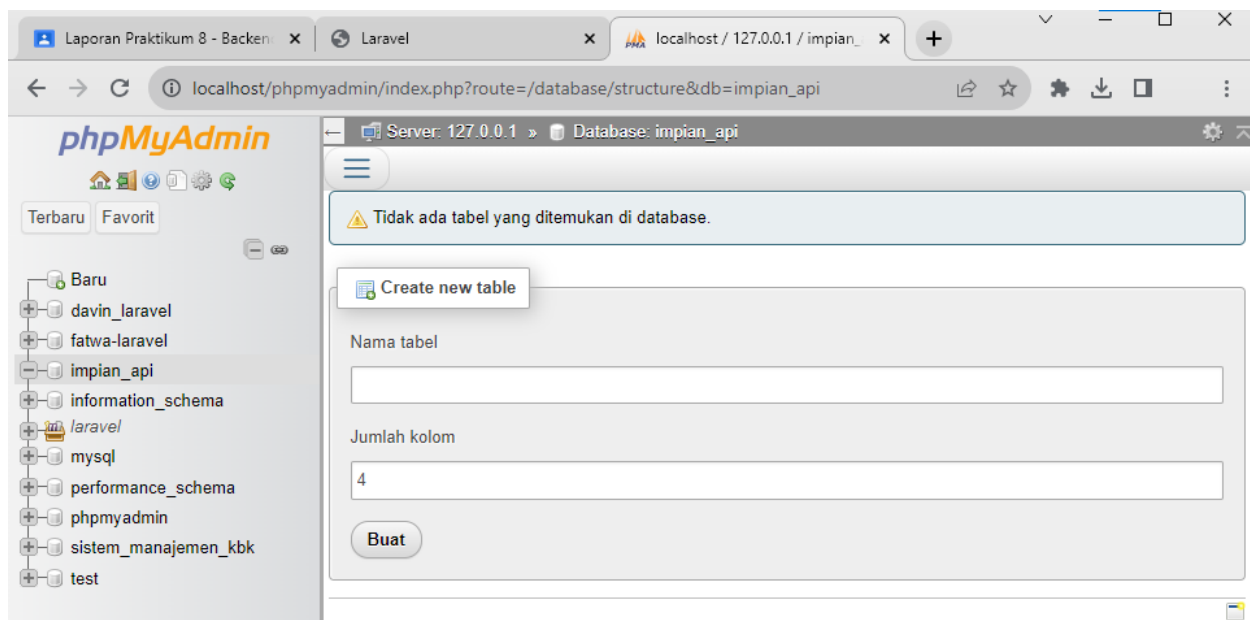
Perintah **php artisan serve** digunakan untuk menjalankan server pengembangan bawaan laravel. Ketika perintah ini dijalankan, laravel akan menggunakan server pengembangan built-in.

3. Setelah itu, saya membuka <http://127.0.0.1:8000>.

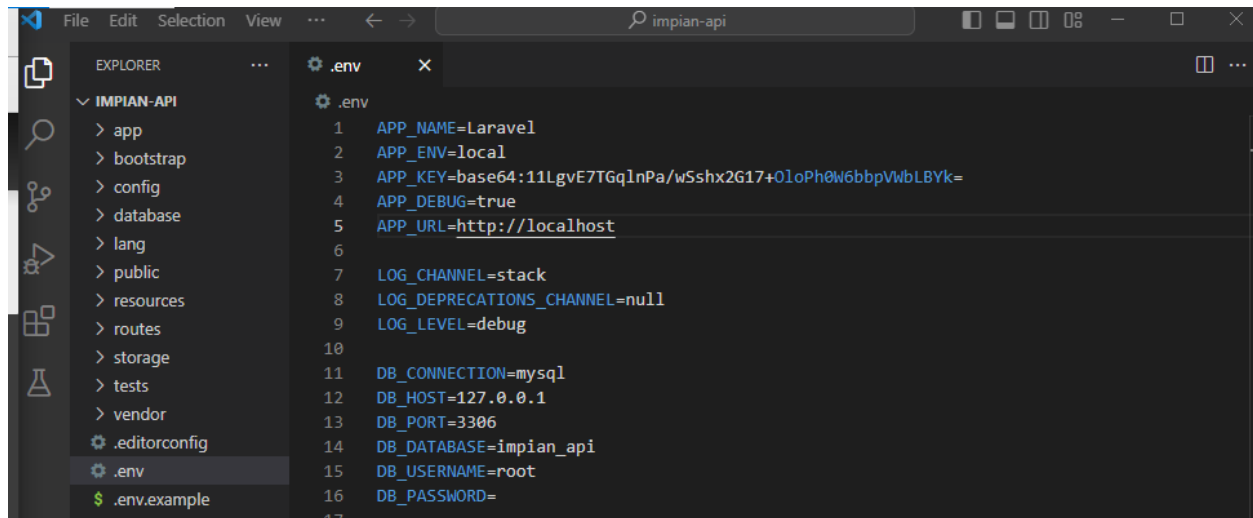


Koneksi Database

1. Saya membuka phpMyAdmin, membuat database dengan nama impian_api.



2. Saya membuka file `.env`, menyesuaikan konfigurasi database username, password, dan nama database.

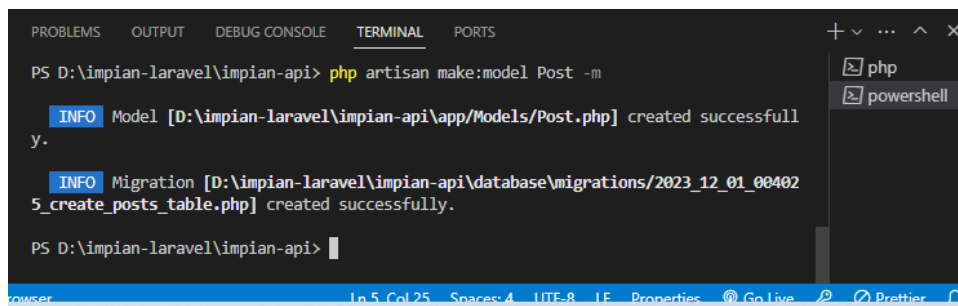


The screenshot shows the Visual Studio Code editor with the `.env` file open. The Explorer sidebar on the left shows the project structure for `IMPIAN-API`, with `.env` selected under the `config` directory. The `.env` file contains the following configuration:

```
1 APP_NAME=Laravel
2 APP_ENV=local
3 APP_KEY=base64:11LgvE7TGqInPa/wSshx2G17+01oPh0w6bbpVwBLBYk=
4 APP_DEBUG=true
5 APP_URL=http://localhost
6
7 LOG_CHANNEL=stack
8 LOG_DEPRECATIONS_CHANNEL=null
9 LOG_LEVEL=debug
10
11 DB_CONNECTION=mysql
12 DB_HOST=127.0.0.1
13 DB_PORT=3306
14 DB_DATABASE=impian_api
15 DB_USERNAME=root
16 DB_PASSWORD=
```

Membuat Model dan Migration

1. Saya membuka CMD dan menjalankan perintah `php artisan make:model Post -m`.



The screenshot shows a terminal window with the following output:

```
PS D:\impian-laravel\impian-api> php artisan make:model Post -m

[INFO] Model [D:\impian-laravel\impian-api\app\Models\Post.php] created successfully.

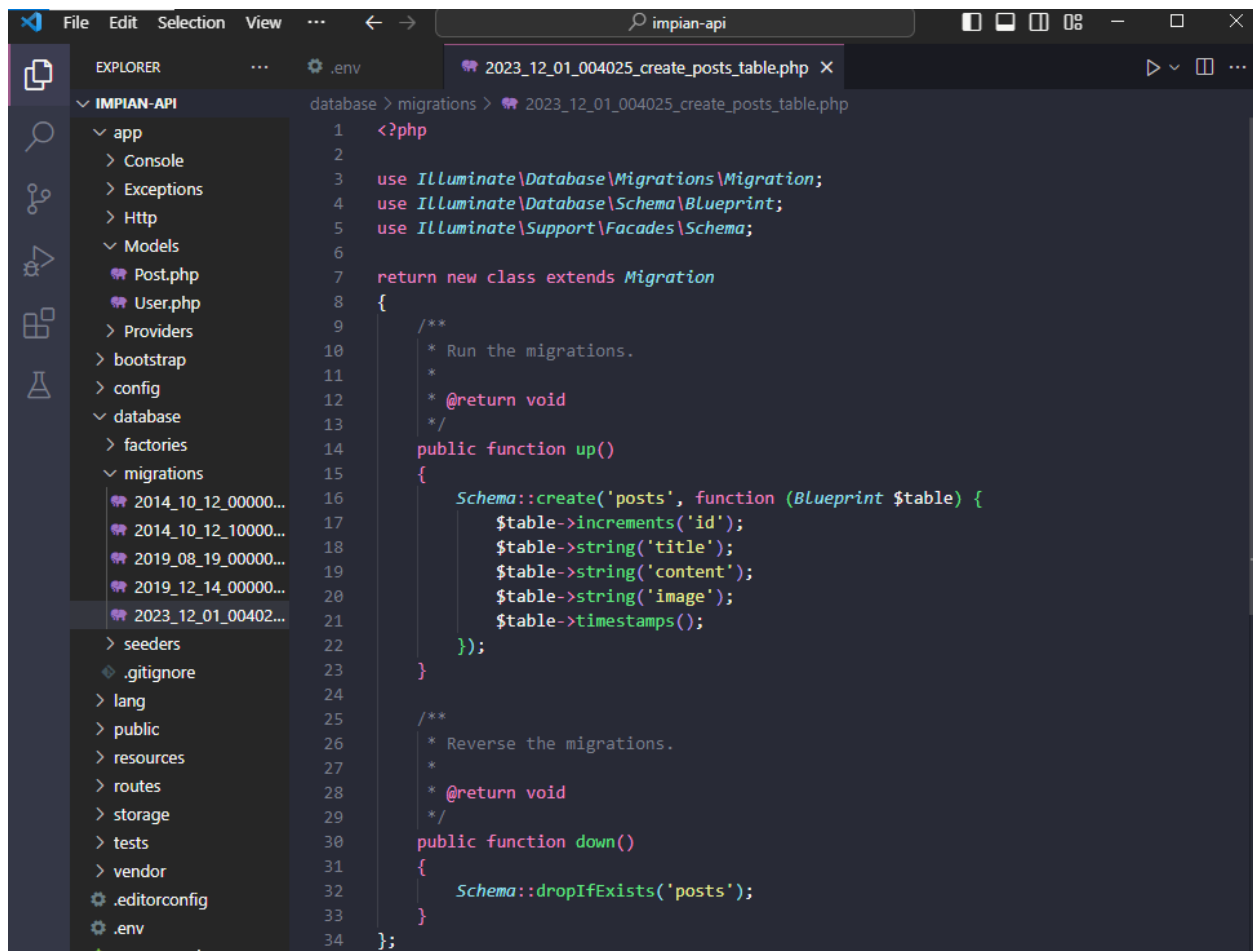
[INFO] Migration [D:\impian-laravel\impian-api\database\migrations\2023_12_01_004025_create_posts_table.php] created successfully.

PS D:\impian-laravel\impian-api>
```

Analisa :

perintah **`php artisan make:model Post -m`** akan membuat file model dengan nama "Post" di direktori `app/Models` dan membuat file migration untuk tabel terkait di direktori `database/migrations`.

2. Saya membuka file migration Post dan mengedit seperti pada gambar di bawah ini.

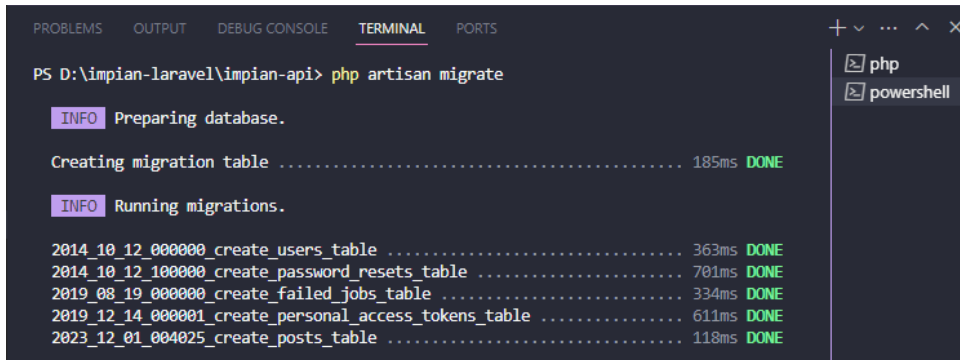
A screenshot of a code editor window titled 'impiant-api'. The left sidebar shows a file explorer with a tree view of the project structure. The 'database' folder is expanded, showing 'migrations'. A file named '2023_12_01_004025_create_posts_table.php' is selected. The main editor area displays the PHP code for this migration file. The code includes the necessary Laravel migration classes and defines an 'up()' method to create a 'posts' table with columns 'id', 'title', 'content', 'image', and 'timestamps'. It also includes a 'down()' method to drop the 'posts' table if it exists.

```
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  return new class extends Migration
8  {
9      /**
10       * Run the migrations.
11       *
12       * @return void
13       */
14      public function up()
15      {
16          Schema::create('posts', function (Blueprint $table) {
17              $table->increments('id');
18              $table->string('title');
19              $table->string('content');
20              $table->string('image');
21              $table->timestamps();
22          });
23      }
24
25      /**
26       * Reverse the migrations.
27       *
28       * @return void
29       */
30      public function down()
31      {
32          Schema::dropIfExists('posts');
33      }
34  };
```

Analisa :

Kode program di atas adalah sebuah Laravel migration file. Dalam method `up()`, sebuah tabel 'posts' dibuat dengan kolom 'id' (auto-increment), 'title', 'content', 'image', dan kolom waktu ('timestamps'). Method `down()` menentukan cara menghapus tabel 'posts' jika migrasi di-"rollback". Secara keseluruhan, kode ini mengelola skema database untuk menciptakan dan menghapus tabel 'posts'.

3. Setelah itu, saya menjalankan perintah php artisan migrate.



```
PS D:\impian-laravel\impian-api> php artisan migrate

[INFO] Preparing database.

Creating migration table ..... 185ms DONE

[INFO] Running migrations.

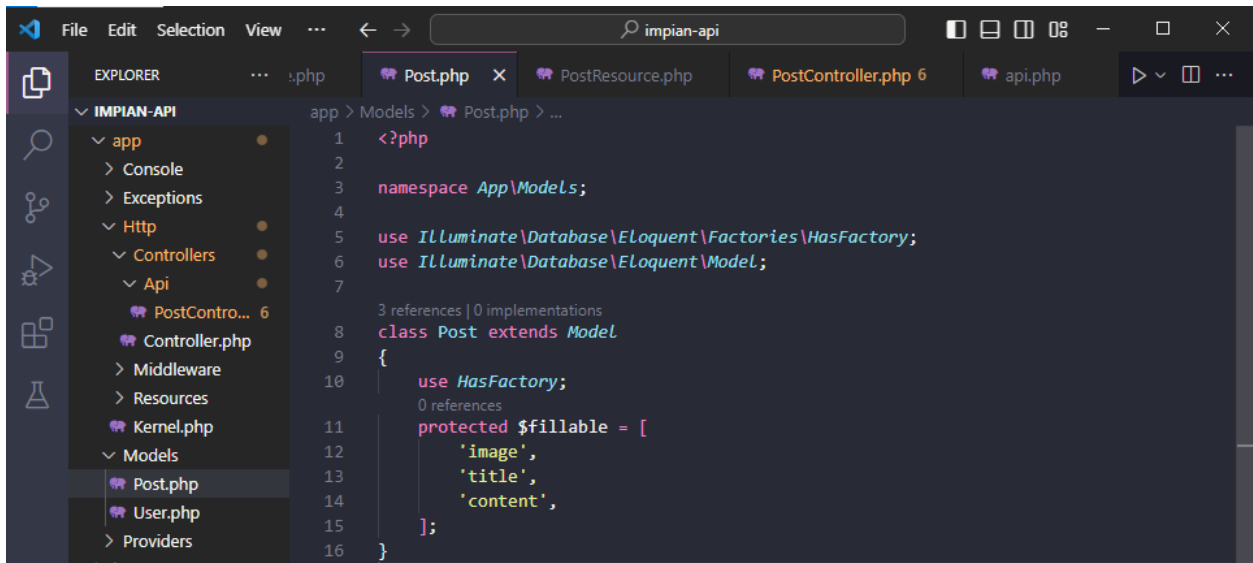
2014_10_12_000000_create_users_table ..... 363ms DONE
2014_10_12_100000_create_password_resets_table ..... 701ms DONE
2019_08_19_000000_create_failed_jobs_table ..... 334ms DONE
2019_12_14_000001_create_personal_access_tokens_table ..... 611ms DONE
2023_12_01_004025_create_posts_table ..... 118ms DONE
```

Analisa :

Perintah **php artisan migrate** dalam Laravel digunakan untuk menjalankan semua file migrasi yang belum pernah dijalankan sebelumnya. Migrasi adalah cara Laravel mengelola skema database, termasuk pembuatan dan penghapusan tabel, serta perubahan lainnya pada struktur database.

Menambahkan Mass Assignment

1. Saya membuka file app/Models/Post.php dan mengubah kode seperti pada gambar di bawah.



```
1 <?php
2
3 namespace App\Models;
4
5 use Illuminate\Database\Eloquent\Factories\HasFactory;
6 use Illuminate\Database\Eloquent\Model;
7
8 class Post extends Model
9 {
10     use HasFactory;
11     0 references
12     protected $fillable = [
13         'image',
14         'title',
15         'content',
16     ];
17 }
```

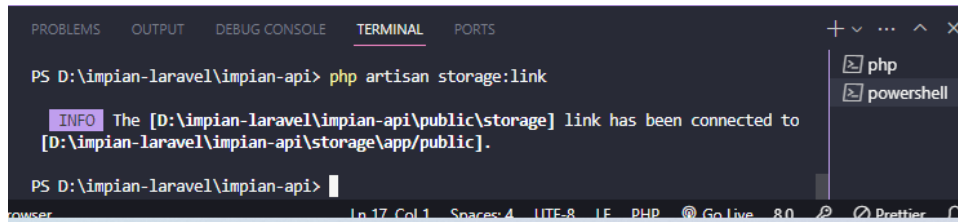
Analisa :

Kode program di atas adalah definisi dari model 'Post' dalam framework Laravel. Model ini memiliki trait 'HasFactory' untuk memudahkan pembuatan data uji, dan properti '\$fillable' yang menentukan kolom-kolom yang dapat diisi secara massal, yaitu 'image', 'title', dan 'content'.

Dengan model ini, Anda dapat dengan mudah berinteraksi dengan tabel terkait dalam basis data menggunakan Eloquent ORM.

Menghubungkan Storage

1. Saya menjalankan perintah berikut di dalam terminal/CMD: `php artisan storage:link`.



```
PS D:\impian-laravel\impian-api> php artisan storage:link

[INFO] The [D:\impian-laravel\impian-api\public\storage] link has been connected to
[D:\impian-laravel\impian-api\storage\app\public].

PS D:\impian-laravel\impian-api>
```

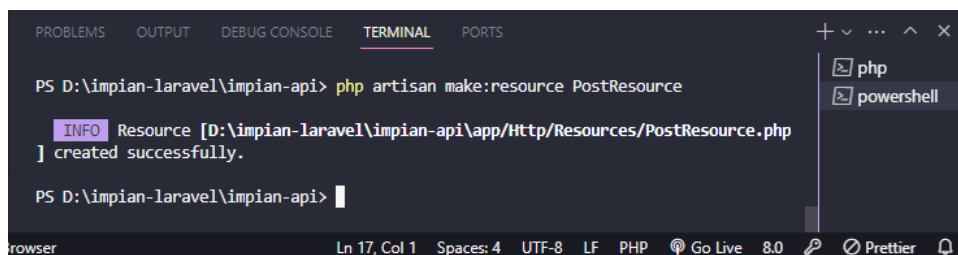
Analisa :

Perintah **php artisan storage:link** dalam Laravel digunakan untuk membuat tautan simbolik (symbolic link) antara direktori penyimpanan publik (public storage) dan direktori publik pada proyek Anda.

API Resources

Membuat API Resources

1. Saya membuka CMD dan menjalankan perintah `php artisan make:resource PostResource`.



```
PS D:\impian-laravel\impian-api> php artisan make:resource PostResource

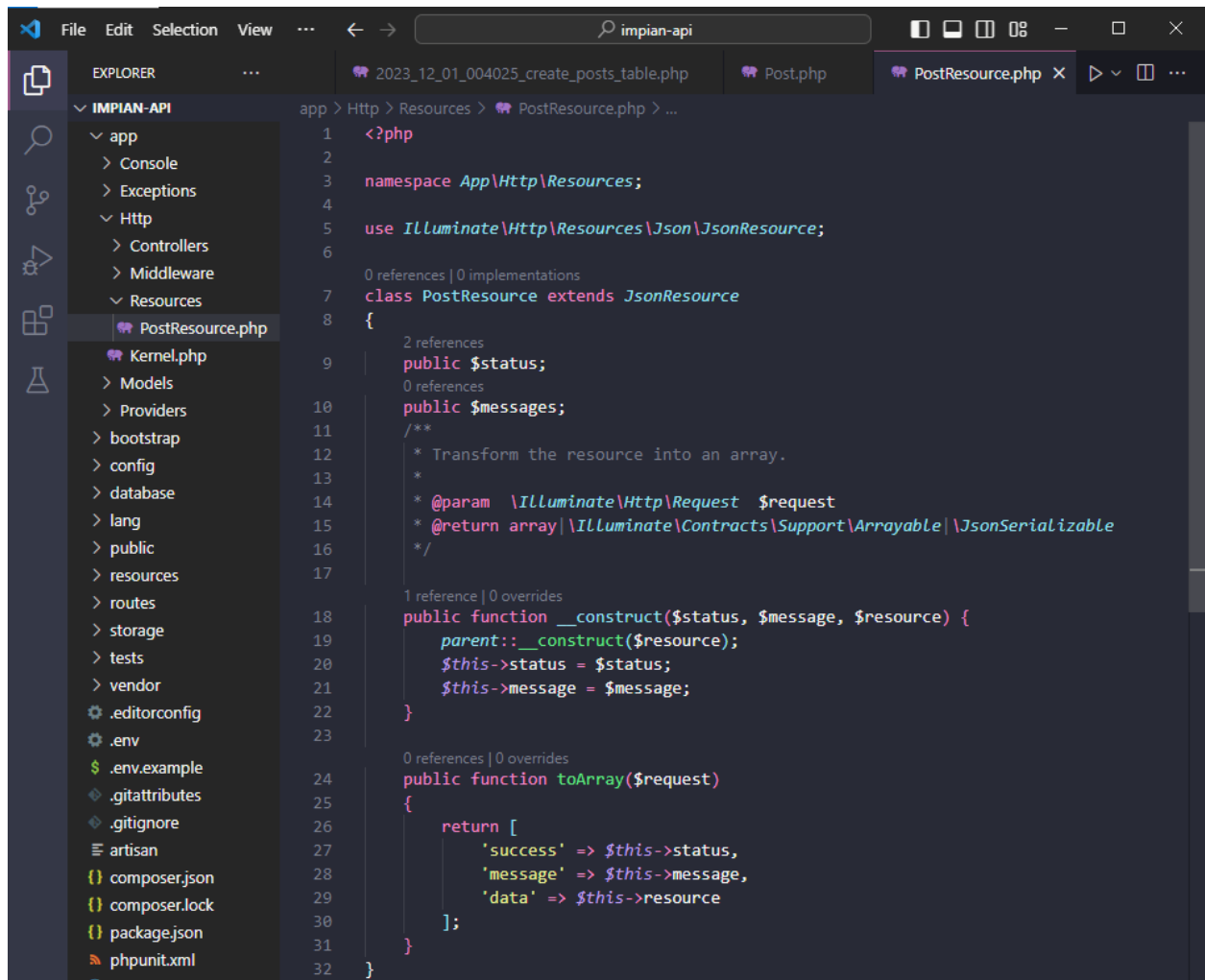
[INFO] Resource [D:\impian-laravel\impian-api\app\Http\Resources\PostResource.php]
created successfully.

PS D:\impian-laravel\impian-api>
```

Analisa :

Perintah **php artisan make:resource PostResource** dalam Laravel digunakan untuk membuat resource class. Resource class digunakan untuk mengubah model dan koleksi model menjadi format yang dapat dihasilkan sebagai respons API atau tampilan JSON yang terstruktur.

2. Saya membuka file app/Http/PostResource dan mengedit seperti pada gambar di bawah ini.



```
1  <?php
2
3  namespace App\Http\Resources;
4
5  use Illuminate\Http\Resources\Json\JsonResource;
6
7  class PostResource extends JsonResource
8  {
9      public $status;
10     public $message;
11     /**
12      * Transform the resource into an array.
13      *
14      * @param  \Illuminate\Http\Request  $request
15      * @return array|\Illuminate\Contracts\Support\Arrayable|\JsonSerializable
16      */
17
18     public function __construct($status, $message, $resource) {
19         parent::__construct($resource);
20         $this->status = $status;
21         $this->message = $message;
22     }
23
24     public function toArray($request)
25     {
26         return [
27             'success' => $this->status,
28             'message' => $this->message,
29             'data' => $this->resource
30         ];
31     }
32 }
```

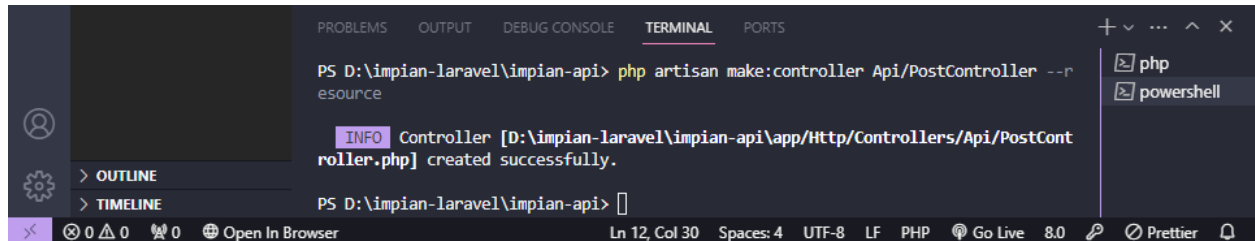
Analisa :

Kode program di atas adalah resource class 'PostResource' dalam Laravel, digunakan untuk mengubah model 'Post' menjadi format JSON yang terstruktur untuk keperluan respons API. Properti 'status' dan 'message' digunakan untuk menyertakan informasi tambahan dalam respons API. Method 'toArray(\$request)' mendefinisikan struktur respons JSON dengan kunci 'success' untuk status, 'message' untuk pesan, dan 'data' untuk data sebenarnya dari model 'Post'. Resource class ini memberikan fleksibilitas dalam menyusun respons API sesuai kebutuhan aplikasi.

Menampilkan Data

Membuat Controller

1. Saya menjalankan perintah `php artisan make:controller Api/PostController --resource`.

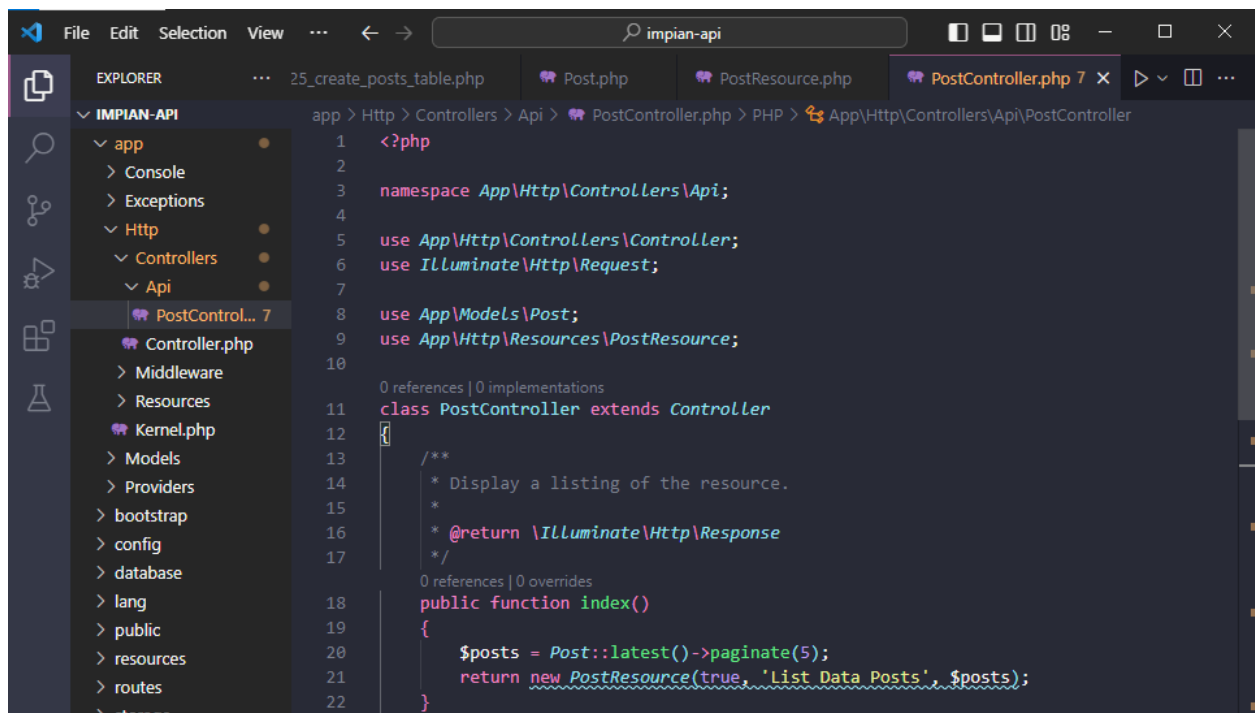


```
PS D:\impian-laravel\impian-api> php artisan make:controller Api/PostController --resource
INFO Controller [D:\impian-laravel\impian-api\app\Http\Controllers\Api/PostController.php] created successfully.
PS D:\impian-laravel\impian-api>
```

Analisa :

Perintah `php artisan make:controller Api/PostController --resource` dalam Laravel digunakan untuk membuat controller yang memiliki implementasi sumber daya atau resourceful controller. Controller yang dihasilkan akan memiliki metode-metode yang umumnya digunakan untuk mengelola sumber daya atau resource, seperti menampilkan daftar, menampilkan satu item, membuat, mengupdate, dan menghapus.

2. Saya mengimpor model, mengimpor Resources, dan menambahkannya pada fungsi index.



```
app > Http > Controllers > Api > PostController.php > PHP > App\Http\Controllers\Api/PostController
1  <?php
2
3  namespace App\Http\Controllers\Api;
4
5  use App\Http\Controllers\Controller;
6  use Illuminate\Http\Request;
7
8  use App\Models\Post;
9  use App\Http\Resources\PostResource;
10
11 0 references | 0 implementations
12 class PostController extends Controller
13 {
14     /**
15      * Display a listing of the resource.
16      *
17      * @return \Illuminate\Http\Response
18      */
19     0 references | 0 overrides
20     public function index()
21     {
22         $posts = Post::latest()->paginate(5);
23         return new PostResource(true, 'List Data Posts', $posts);
24     }
25 }
```

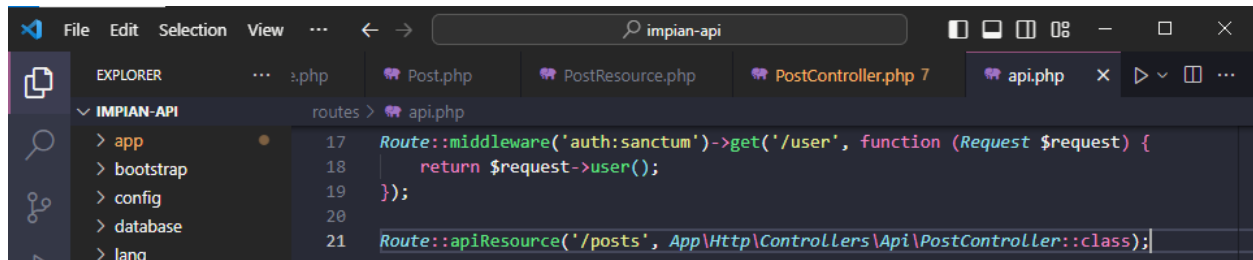
Analisa :

Controller ini mencakup metode-metode seperti `index()` untuk menampilkan daftar post, `store()` untuk menyimpan post baru dengan validasi input, `show()` untuk menampilkan detail post, `update()` untuk memperbarui post dengan validasi, dan `destroy()` untuk menghapus post.

Dalam metode `store()` dan `update()`, terdapat penggunaan Validator untuk validasi input, serta pengelolaan file gambar menggunakan Laravel Storage. Metode `index()` menggunakan model Post untuk mengambil dan menampilkan daftar post terbaru dengan paginasi.

Membuat Routing

1. Saya membuka file `routes/api.php` dan menambahkannya.

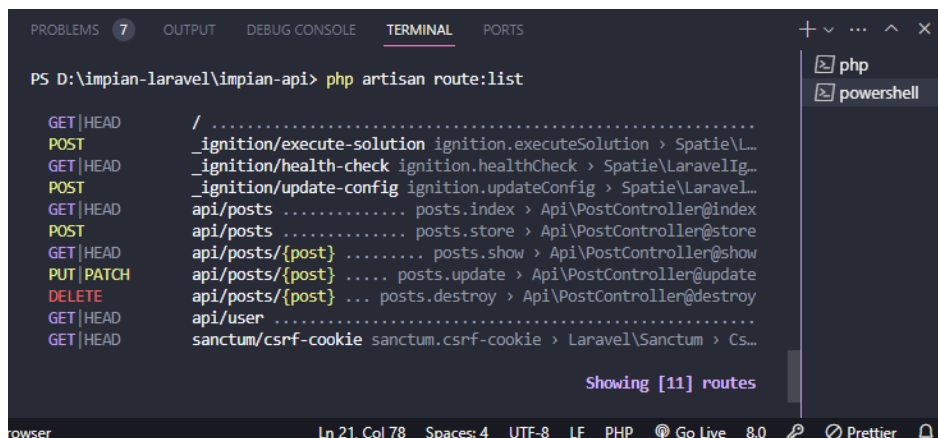


```
File Edit Selection View ... < > impian-api
EXPLORER routes > api.php
IMPIAN-API
  > app
  > bootstrap
  > config
  > database
  > lang
  routes > api.php
    17 Route::middleware('auth:sanctum')->get('/user', function (Request $request) {
    18     return $request->user();
    19 });
    20
    21 Route::apiResource('/posts', App\Http\Controllers\Api\PostController::class);
```

Analisa :

Dengan menggunakan `apiResource`, Laravel secara otomatis membuat rute untuk metode `index`, `store`, `show`, `update`, dan `destroy` di dalam controller `PostController`. Sebagai contoh, rute GET ke `'/posts'` akan dipetakan ke metode `index()` dalam `PostController`, sedangkan rute POST ke `'/posts'` akan dipetakan ke metode `store()`.

2. Untuk memastikan bahwa route telah berhasil ditambahkan, saya menjalankan perintah berikut di dalam terminal/CMD: `php artisan route:list`.



```
PROBLEMS 7 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\impian-laravel\impian-api> php artisan route:list

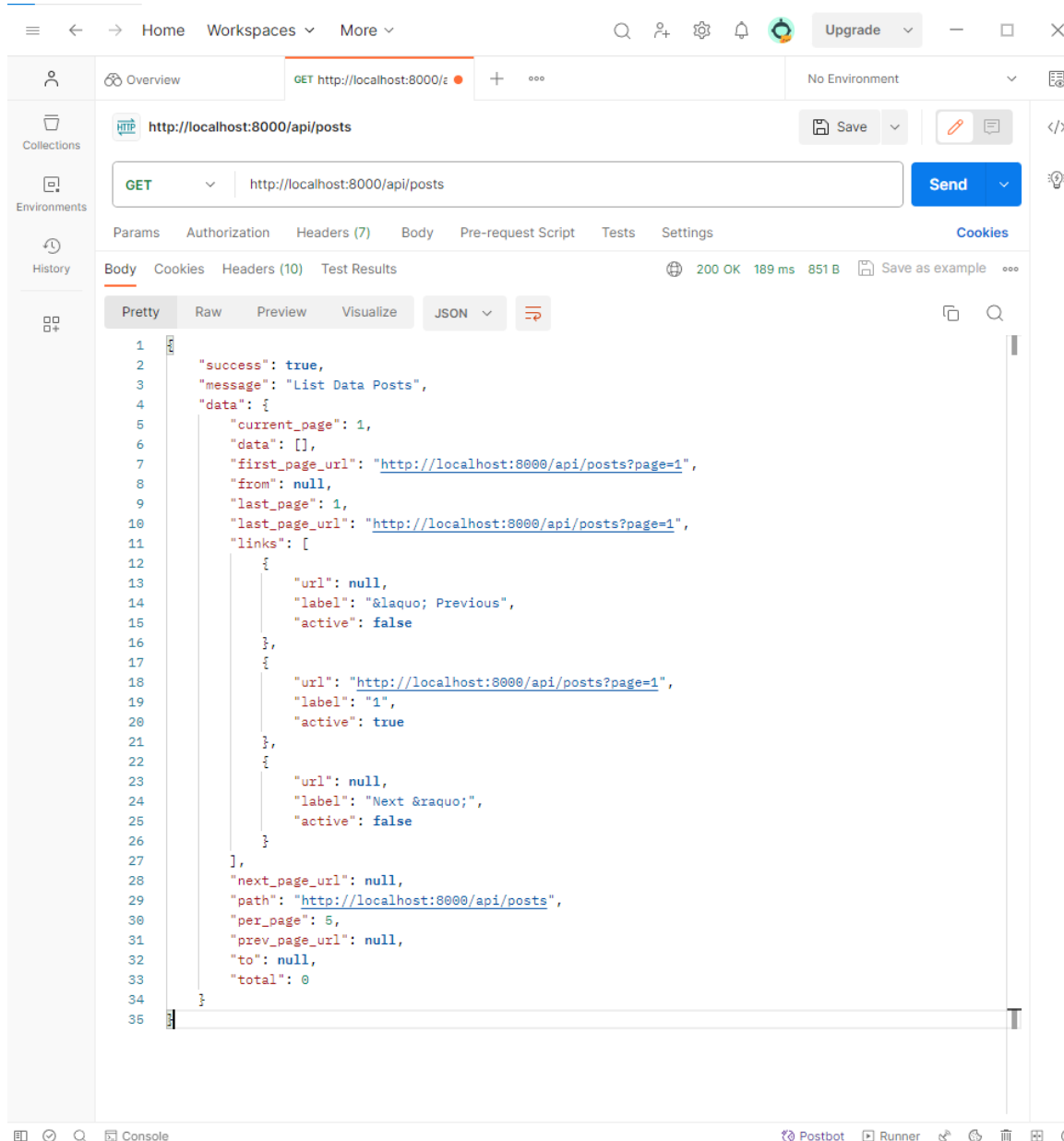
GET|HEAD / .....
POST _ignition/execute-solution ignition.executeSolution > Spatie\L...
GET|HEAD _ignition/health-check ignition.healthCheck > Spatie\LaravelIg...
POST _ignition/update-config ignition.updateConfig > Spatie\Laravel...
GET|HEAD api/posts ..... posts.index > Api\PostController@index
POST api/posts ..... posts.store > Api\PostController@store
GET|HEAD api/posts/{post} ..... posts.show > Api\PostController@show
PUT|PATCH api/posts/{post} ..... posts.update > Api\PostController@update
DELETE api/posts/{post} ... posts.destroy > Api\PostController@destroy
GET|HEAD api/user .....
GET|HEAD sanctum/csrf-cookie sanctum.csrf-cookie > Laravel\Sanctum > Cs...

Showing [11] routes
```

Analisa :

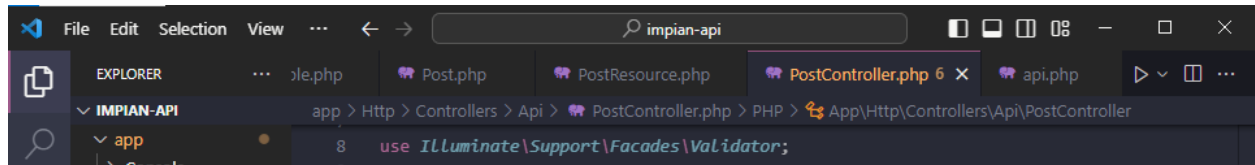
Perintah **php artisan route:list** dalam Laravel digunakan untuk menampilkan daftar seluruh rute (routes) yang terdaftar dalam aplikasi Laravel Anda. Ketika perintah ini dijalankan, Anda akan melihat output berupa tabel yang mencakup informasi seperti method HTTP, URI (Uniform Resource Identifier), nama rute, controller yang menangani rute, dan beberapa informasi tambahan.

3. Saya membuka Postman dan mengakses route URL: <http://localhost:8000/api/posts>.



Insert Data

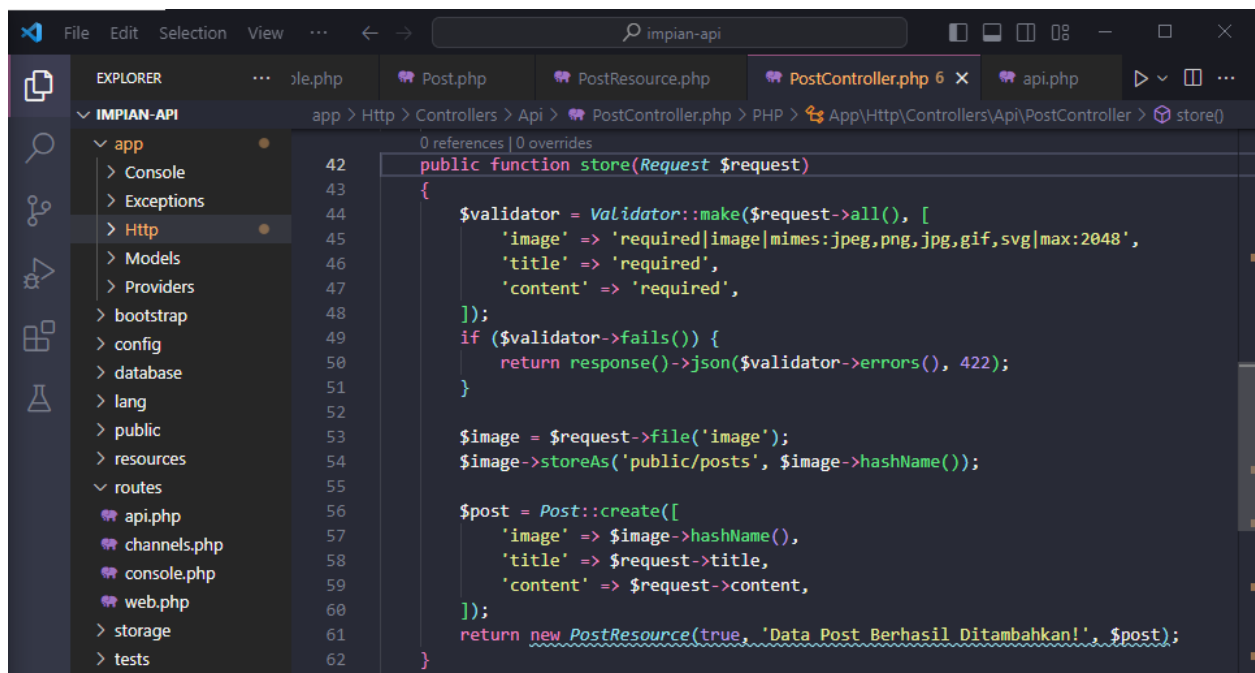
1. Saya menambahkan validator di dalam file PostController.php.



Analisa :

`use Illuminate\Support\Facades\Validator;` digunakan untuk mengimpor fasad Validator dari Laravel. Fasad (Facade) adalah sebuah desain pola yang memungkinkan kita mengakses objek di dalam kontainer layanan Laravel tanpa harus membuat instance langsung.

2. Saya menambahkan kode di bawah ini pada fungsi store.

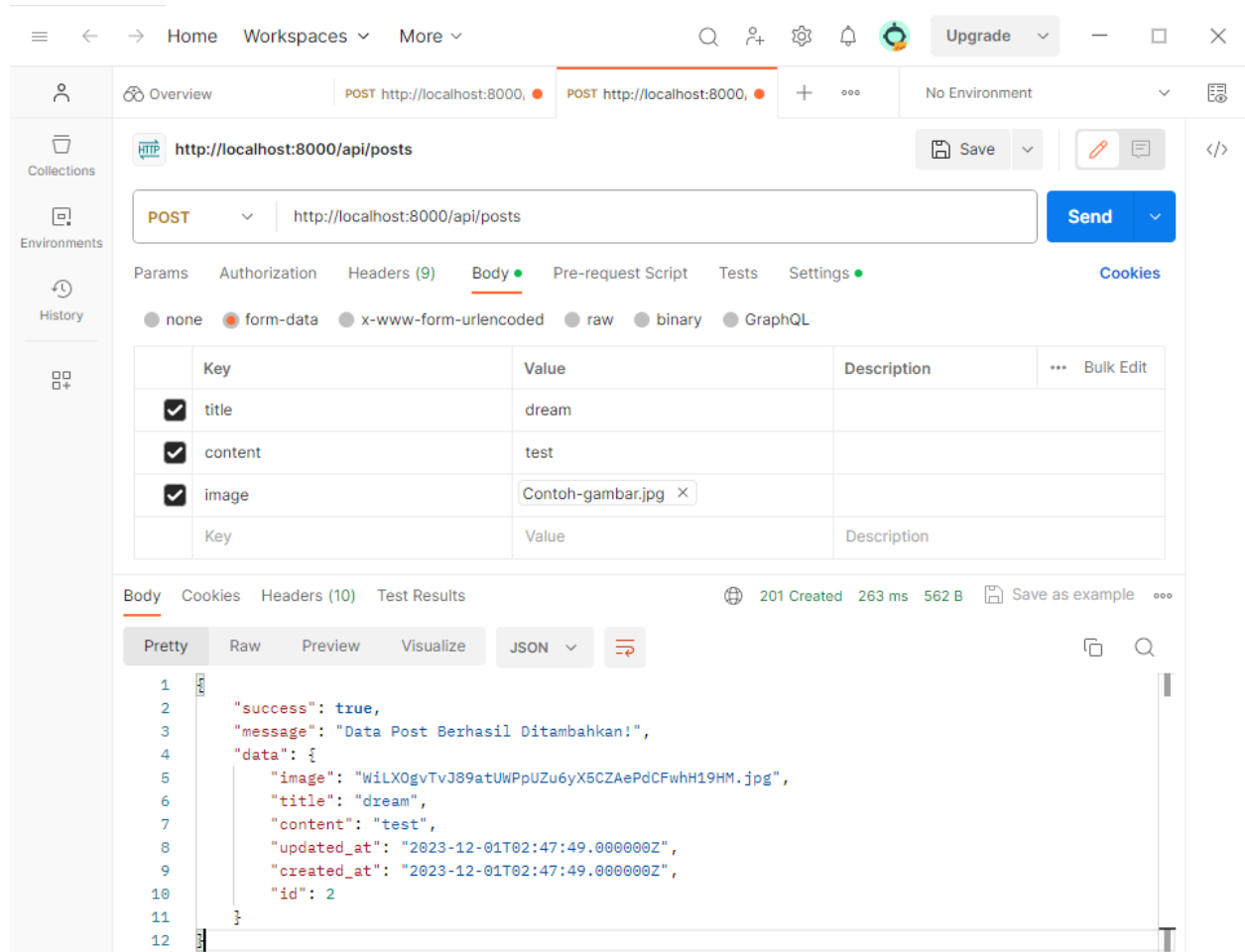


Analisa :

Fungsi dari function **store** ini adalah untuk menyimpan data post baru dalam database. Pertama-tama, data dari permintaan validasi dengan aturan tertentu, termasuk memastikan gambar memiliki format dan ukuran yang sesuai. Jika validasi gagal, dikembalikan respons JSON dengan pesan kesalahan.

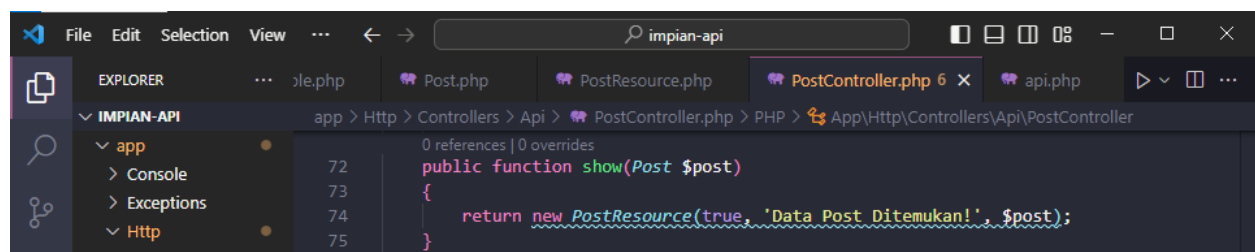
Jika validasi berhasil, gambar diambil, disimpan dalam direktori public/posts, dan data post baru dibuat dan disimpan ke dalam database. Terakhir, respons JSON dikembalikan dengan pesan sukses dan informasi post yang baru ditambahkan.

3. Saya menguji menggunakan Postman.



Menampilkan Data Berdasarkan ID

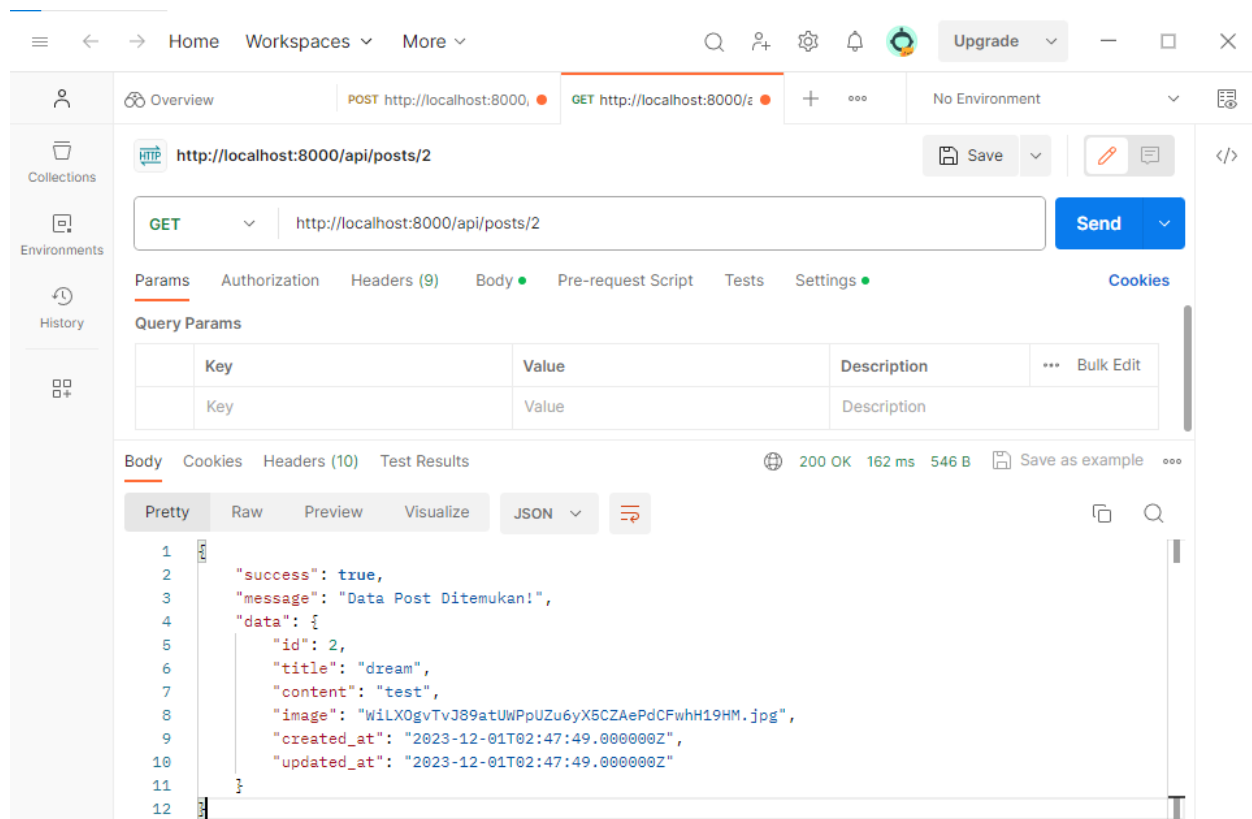
1. Saya menambahkan kode di bawah ini pada file PostController.php.



Analisa :

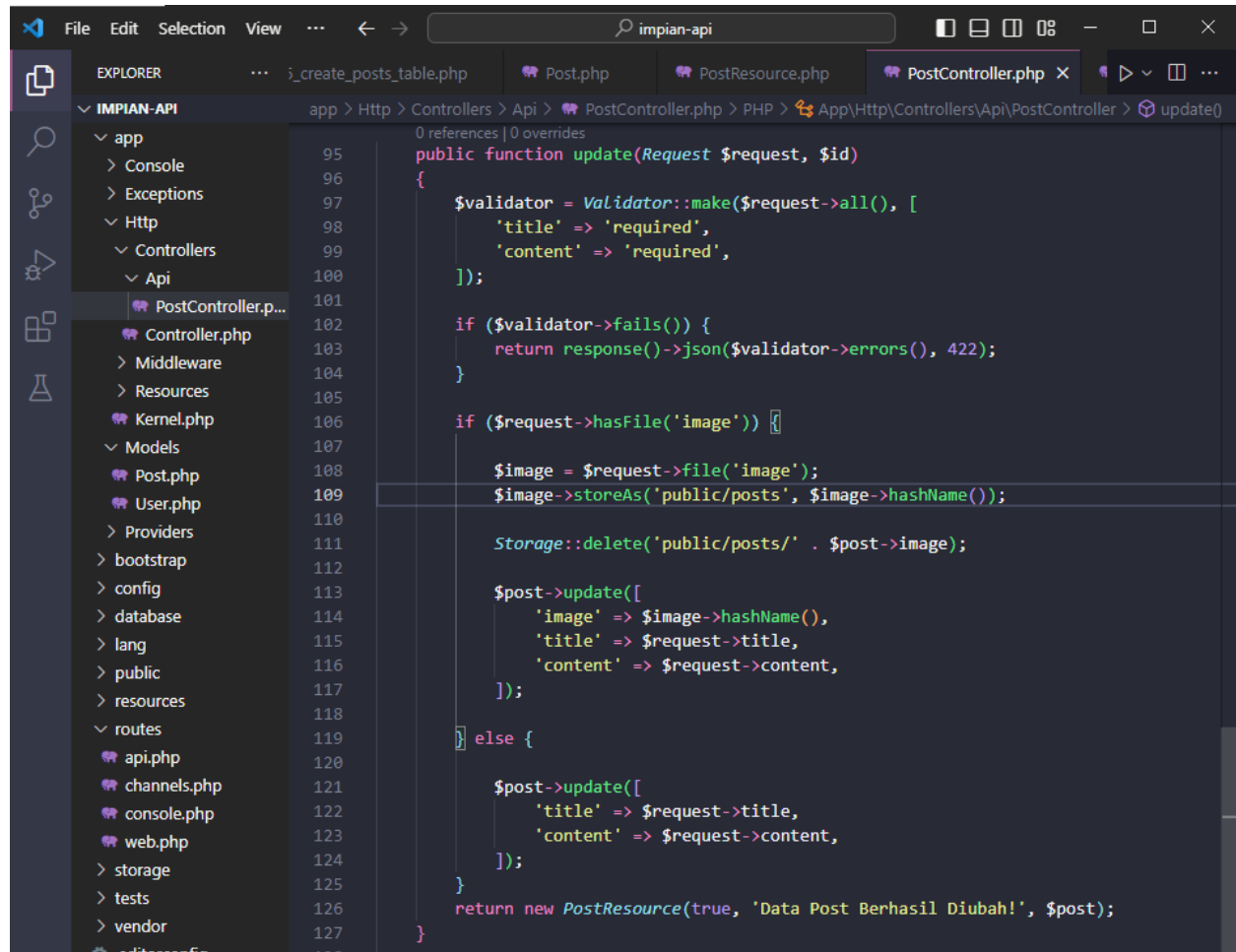
Fungsi dari function **show** ini adalah untuk menampilkan data post tertentu berdasarkan model Post. Metode ini mengembalikan respons JSON yang berisi informasi sukses dan data post yang ditemukan.

2. Saya menguji dengan Postman menggunakan metode GET dengan endpoint URL `http://localhost:8000/api/posts/:id`.



Update Data

1. Saya menambahkan kode di bawah ini pada file PostController.php.

The image is a screenshot of a code editor window titled 'impian-api'. The left sidebar shows a file explorer with a tree structure: 'app' (containing Console, Exceptions, Http, Controllers, Api, Middleware, Resources, Kernel.php, Models, Post.php, User.php, Providers, bootstrap, config, database, lang, public, resources, routes, api.php, channels.php, console.php, web.php, storage, tests, vendor, and editorconfig). The 'Api' folder is expanded, showing 'PostController.php'. The main editor area displays the code for the 'update()' method in 'PostController.php'. The code is as follows:

```
0 references | 0 overrides
public function update(Request $request, $id)
{
    $validator = Validator::make($request->all(), [
        'title' => 'required',
        'content' => 'required',
    ]);

    if ($validator->fails()) {
        return response()->json($validator->errors(), 422);
    }

    if ($request->hasFile('image')) {
        $image = $request->file('image');
        $image->storeAs('public/posts', $image->hashName());

        Storage::delete('public/posts/' . $post->image);

        $post->update([
            'image' => $image->hashName(),
            'title' => $request->title,
            'content' => $request->content,
        ]);
    } else {
        $post->update([
            'title' => $request->title,
            'content' => $request->content,
        ]);
    }

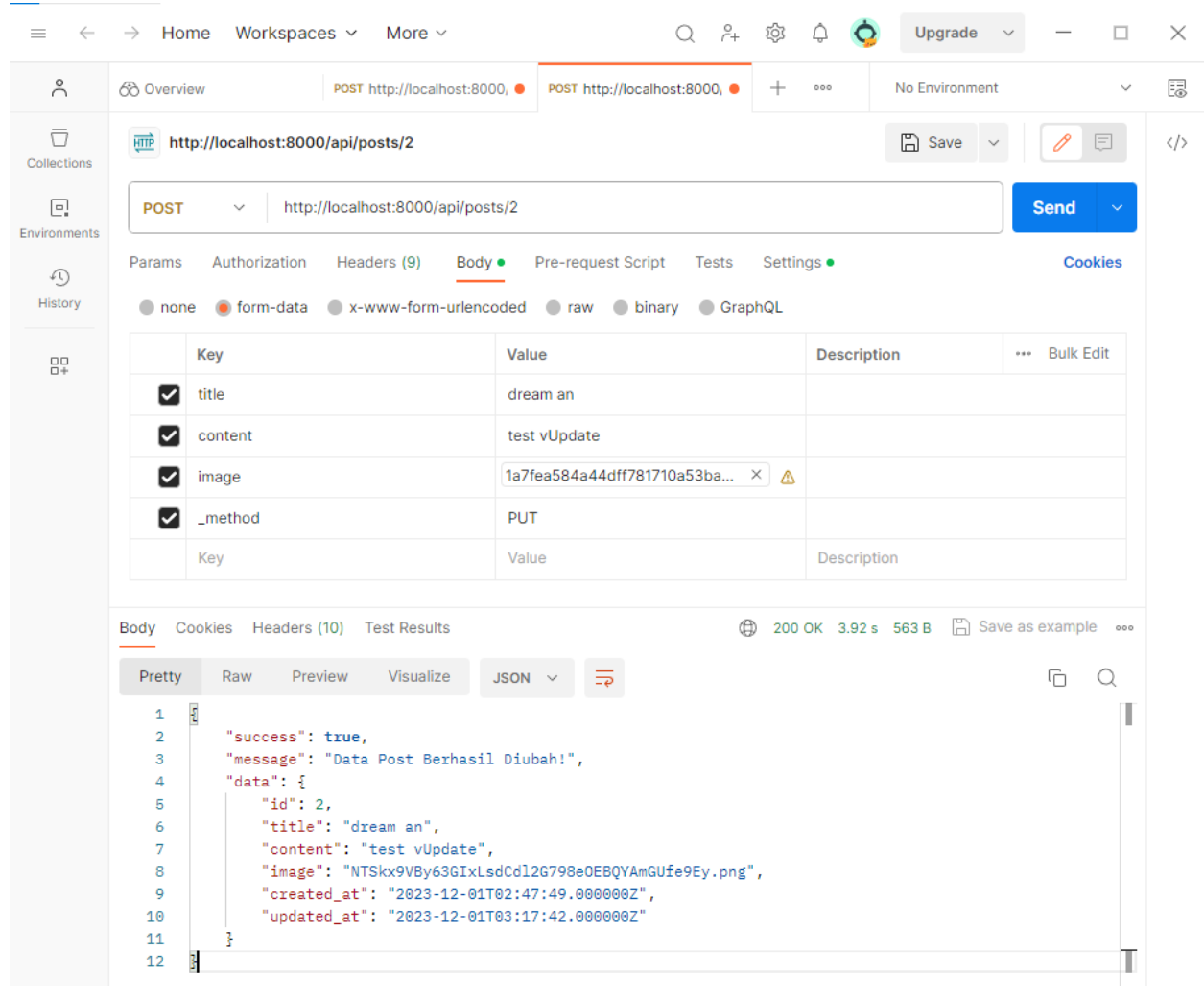
    return new PostResource(true, 'Data Post Berhasil Diubah!', $post);
}
```

Analisa :

Fungsi dari function **update** ini bertujuan untuk mengelola pembaruan data post berdasarkan ID yang diberikan. Metode melakukan validasi pada bidang `title` dan `content`. Jika validasi berhasil, post diidentifikasi berdasarkan ID, dan jika ada file gambar baru, gambar lama digantikan.

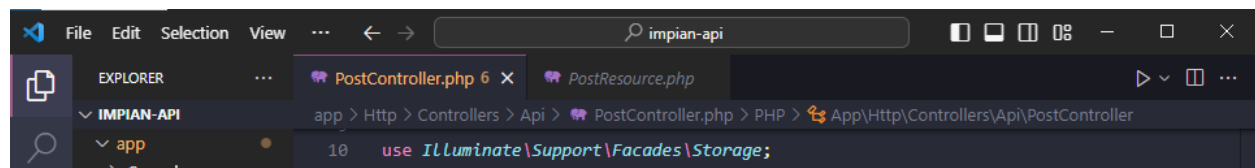
Setelah itu, data post diperbarui berdasarkan input dari permintaan. Metode mengembalikan respons JSON dengan pesan sukses dan data post yang telah diperbarui.

2. Saya menguji dengan Postman menggunakan metode POST dengan endpoint URL `http://localhost:8000/api/posts/:id`.



Delete Data

1. Saya menambahkan library storage.

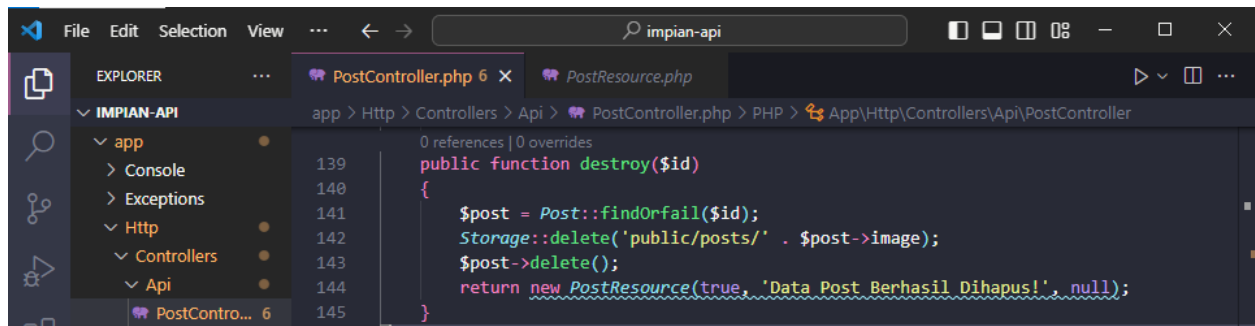


Analisa :

`use Illuminate\Support\Facades\Storage;` digunakan untuk mengimpor (menggunakan) fasad Storage dari framework Laravel. Fasad ini memberikan akses kepada pengembang untuk

berinteraksi dengan sistem penyimpanan file Laravel tanpa harus membuat instance langsung dari kelas Storage.

2. Saya menambahkan kode di bawah ini pada file PostController.php.



```
0 references | 0 overrides
public function destroy($id)
{
    $post = Post::findOrFail($id);
    Storage::delete('public/posts/' . $post->image);
    $post->delete();
    return new PostResource(true, 'Data Post Berhasil Dihapus!', null);
}
```

Analisa :

Fungsi dari function **destroy** ini adalah untuk mengelola penghapusan data post berdasarkan ID. Metode ini mencari post berdasarkan ID, menghapus file gambar terkait dari penyimpanan, kemudian menghapus post dari database. Metode mengembalikan respons JSON yang menyatakan bahwa penghapusan data post telah berhasil dilakukan, bersama dengan pesan sukses.

3. Saya menguji dengan Postman menggunakan metode DELETE dengan endpoint URL <http://localhost:8000/api/posts/:id>.

